FINJARVIS: An AI Based Personal Finance Manager with Expense Categorisation, Spending Forecast and Health Score

MEKALA KALYAN
Roll No. 23K91A05F0
Dept. of CSE
TKR College of Engineering & Technology
Hyderabad
[EMAIL]

RAJANALA SAI HEMANTH
Roll No. 23K91A05K3
Dept. of CSE
TKR College of Engineering & Technology
Hyderabad
[EMAIL]

MIDDALA SURYA TEJA REDDY
Roll No. 23K91A05F2
Dept. of CSE
TKR College of Engineering & Technology
Hyderabad
[EMAIL]

MANGA AKASH
Roll No. 23K91A05E5
Dept. of CSE
TKR College of Engineering & Technology
Hyderabad
[EMAIL]

N PADMAVATHI
[DESIGNATION]
(project guide)
Dept. of CSE
TKR College of Engineering & Technology
Hyderabad
[EMAIL]

***Abstract*---**Most budgeting tools report what was spent but neither predict nor explain it, and manual entry is tedious. This paper presents FINJARVIS, a web application in which a user records or imports transactions, a TF-IDF and logistic-regression model labels each one, a linear trend forecasts next month's spending, six weighted components give a 0-100 financial health score, and a rule-routed assistant answers questions from the user's own records. The system has 33 API routes and 6 tables and passes 38 unit tests and 49 end-to-end checks. On a 198-description probe set the categoriser reaches 71.7 % accuracy, but only 18.2 % on merchants it has not seen, because it learns from just 51 examples. On a six-month demo history the linear trend is not better than a plain average, and a simulation shows that it needs about seven months of data before it is. The assistant answered 14 of 20 answerable questions. Anomaly detection, reinforcement-learning budgeting and deep-learning forecasting from the base paper are not implemented.

***Index Terms*---**budgeting, expense categorisation, financial health score, forecasting, logistic regression, personal finance, TF-IDF.

I. INTRODUCTION

People now hold salaries, cards, UPI payments, recurring bills and investments at once, and managing them well matters: financial literacy is linked to saving, investment and retirement planning [2], and low literacy is reported among young adults in particular [3]. In practice most people either keep no record or keep one by hand. A spreadsheet or a ledger shows what has already been spent, takes time to maintain, and says nothing about the month ahead; commercial applications add automation but are closed, paid and not adaptable to one household's commitments. The base paper of this project [1] proposes four models in one assistant (BERT categorisation, Q-learning budgeting, LSTM forecasting and Isolation Forest anomaly detection) and reports high scores, but on synthetic or privately collected data, and without a runnable application that others can inspect.

This paper makes four contributions. (1) FINJARVIS, a tested full-stack platform (FastAPI, React, SQLite; 33 routes, 6 tables) for accounts, transactions, CSV and Excel import, budgets with alerts, goals, investments, reports and an assistant. (2) Three transparent AI modules: a TF-IDF and logistic-regression categoriser that retrains on a user's corrections, a least-squares trend forecast, and a six-component health score. (3) Verification with 38 pytest tests, 49 end-to-end checks and measured response times. (4) An honest measurement of where the simple modules fail: on unseen merchants, on short histories and on questions outside five intents. Section II reviews related work, Section III the method, Section IV the results and Section V concludes.

A. System Hardware

All measurements were taken on one machine: Intel Core i5-12500 (6 cores, 12 threads), 15.7 GB RAM, Intel UHD 770 graphics (no discrete GPU, none used), Windows 11 Pro, with other programs running.

B. Software Requirements

Python 3.11.9, FastAPI 0.159.1, SQLAlchemy 2.1.6 [4] on SQLite, scikit-learn 1.9.1 [5], pandas 3.0.6, NumPy 2.4.6, pydantic 2.14.1, passlib 1.7.4 with bcrypt 4.0.1 [6], ReportLab 5.1.0 and openpyxl 3.1.5 for reports, React 18.3 with Vite 8, Tailwind 4, Recharts and TanStack Query on Node 24.19.0, pytest 8.2.0 and Playwright with Chrome for tests and screenshots.

II. LITERATURE SURVEY

Base paper. Kharat et al. [1] build a Smart Personal Finance Assistant from four modules. A bert-base-uncased model is fine-tuned to classify transaction descriptions (128 tokens, learning rate 5e-5, batch 8, ten epochs, 80/20 split), with the category set widened from 16 to 50 and SMOTE [7] used against class imbalance; a tabular Q-learning [8] agent with states built from income, fixed and variable expenses and a savings goal, a reward of +1 or -1, alpha 0.1, gamma 0.95 and epsilon decaying from 0.995 to 0.01 over 300 episodes recommends one of three savings levels; an LSTM [9] with 64 units trained on 12-month savings sequences forecasts the next savings value; and an Isolation Forest [10] with 300 trees and a contamination of 0.015 flags unusual transactions from savings and expense ratios. The modules sit behind a Streamlit interface. Reported results are 92.3 % accuracy, 90.8 % precision, 91.5 % recall and 91.1 % F1 for

categorisation, RMSE 137.2, MAE 89.5 and R2 0.63 for the savings forecast, and 93 %, 94 % and 94 % precision, recall and F1 for anomalies, plus improvements of 42 %, 31 %, 29 % and 27 % over rule-based, static, ARIMA and manual baselines. The data are synthetic or collected from financial APIs and labelled by hand, are not released, and the unit of the forecast errors is not stated.

Related work. For text categorisation, term weighting with TF-IDF [11] is the classical, inspectable baseline, and pre-trained transformers such as BERT [12] and its financial variant FinBERT [13] add context at the cost of data and compute. For forecasting, error measures such as MAPE need care [14], and the M3 comparison of Makridakis et al. [15] found that simple statistical methods were at least as accurate as the machine-learning methods tested, which is relevant when a user has only a few months of history. Recent personal-finance systems combine such models: FinSenseAI [16] joins XGBoost, an LSTM and SHAP explanations for expense prediction, and a leakage-detection system [17] uses an Isolation Forest and K-Means on synthetic transactions; a data-generation framework [18] fine-tunes an 8-billion-parameter language model for personal-finance advice, and a design study [3] builds a gamified mobile app for young adults. Table I states what each work reports, using only what its text says.

TABLE I
RELATED WORK COMPARED

Work	Approach	Data	Reported result	Limitation
Kharat et al. [1]	BERT, Q-learning, LSTM, Isolation Forest; Streamlit	Synthetic and labelled real data, 50 categories	Accuracy 92.3 %; R2 0.63; anomaly F1 94 %	Data not released; no deployed app
Dilip, Janaki [16]	XGBoost + LSTM + SHAP expense prediction	Real transaction table, size not stated	No numeric result in the text read	Metrics listed, not reported
Abiya et al. [17]	Isolation Forest + K-Means; subscription and balance forecast	Synthetic transactions	Better than rule-based budgeting (no metrics)	Synthetic data only
Theerthala [18]	Fine-tuned 8B LLM advisor on generated reasoning data	19k synthetic samples	Comparable to 14-32B models at about 80 % lower cost	LLM-judged; no transaction pipeline
To [3]	Goal-directed design and gamification for a mobile app	Survey 18-30; 10 card-sort, 6 test users	Usability study, no model results	Small samples; no forecasting

Positioning. Against the base paper, FINJARVIS keeps the idea of one assistant over categorisation, budgeting and forecasting but changes every module. A BERT classifier over 50 categories becomes a TF-IDF and logistic-regression model over 11 categories; Q-learning budget recommendation becomes user-set monthly budgets with alerts at 75, 90 and 100 % of the limit and rule-based suggestions; the LSTM savings forecast becomes a least-squares trend over monthly expense totals; and the Isolation Forest has no counterpart, the nearest feature being the single largest expense in a chosen window. In place of a Streamlit demonstration there is a login-protected web application, and the product adds import with preview, goals, a manual investment portfolio, PDF, Excel and CSV reports, a financial health score and a rule-based assistant. These choices trade accuracy on hard inputs for a system that can be run, read and tested by one team on a laptop.

Gap. The works above either report scores on data that others cannot inspect, report no numbers, or study design without models, and none of them gives a runnable application together with a measurement of its simple modules on unseen input. FINJARVIS provides the application and the measurements; it deliberately implements simpler models than the base paper, and Section IV states what that costs.

III. METHODOLOGY

A. Architecture and Implementation

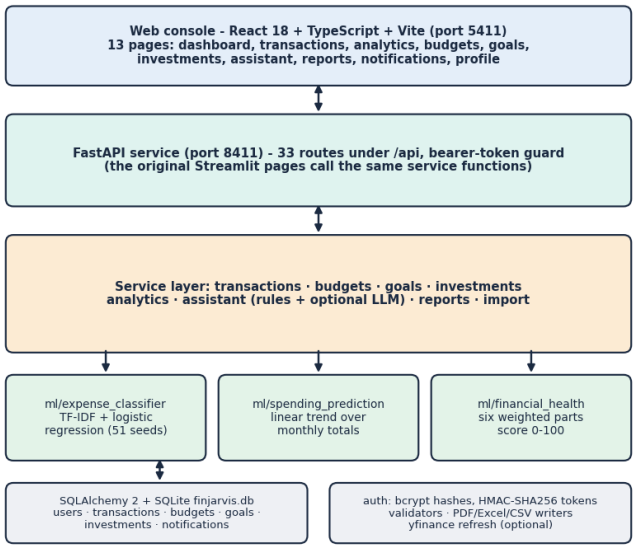


Fig. 1. FINJARVIS architecture: React console, FastAPI service with 33 routes, a service layer, three small ML modules and a SQLite database.

The console (Vite, port 5411) has thirteen pages and calls a FastAPI service (port 8411) whose routes hold no business logic: each calls the same service function that the original Streamlit pages call, so both interfaces share one SQLite file and one set of tests. Requests are validated by pydantic and protected by a bearer-token dependency; data are scoped by a user_id on every table and query. Passwords need eight characters with upper case, lower case, a digit and a symbol, and are stored as bcrypt hashes; tokens are HMAC-SHA256 signed and expire after seven days. Six tables (users, transactions, budgets, financial_goals, investments, notifications) are mapped with SQLAlchemy, with cascading deletes. Reports are written as PDF, Excel and CSV. An optional language-model layer is used only when a key is configured and is not part of any measurement here. The design follows the REST style [19]; the choice of SQLite keeps installation to one file, at the price of a single writer. Fig. 2 summarises the flow of data through the modules described next.

B. Expense Categoriser

The categoriser is a scikit-learn pipeline: word 1-2-gram TF-IDF [11] followed by multinomial logistic regression, trained on a built-in list of 51 description and category pairs in 11 categories (3 to 6 per category). For a description x with feature vector phi(x), the predicted category is

$$k^* = \operatorname{argmax}_k P(k \mid x), \quad P(k \mid x) = \exp(w_k \cdot \mathrm{phi}(x) + b_k) \,/\, \Sigma_j \exp(w_j \cdot \mathrm{phi}(x) + b_j). \qquad (1)$$

When a user adds a transaction with category 'Auto-detect' the label is stored; if the user chooses a different category, the model is retrained on the seed list, the user's whole history and the correction. A keyword table is used only if the stored model cannot be loaded, and the model never outputs 'Other'. Retraining takes about 5 ms.

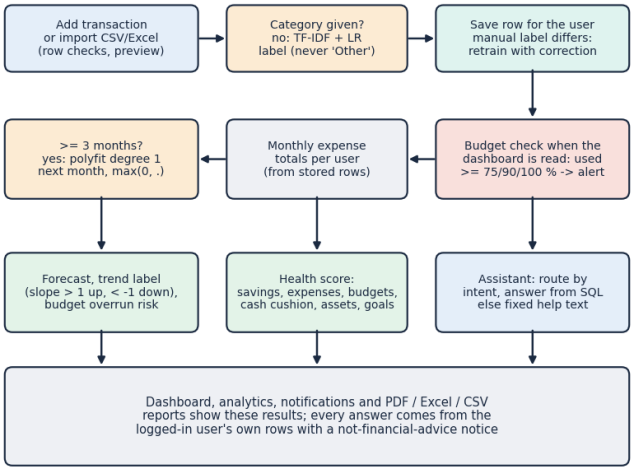


Fig. 2. Main flow of FINJARVIS: transaction entry, automatic category, budget alert, monthly totals, forecast, health score and assistant.

C. Forecast and Health Score

Monthly expense totals of n months (indexed 0 to n-1) are fitted by least squares and extrapolated one step, clipped at zero:

$$\hat{y}_{next} = \max(0,\, a \cdot n + b), \quad (a, b) = \operatorname{argmin} \Sigma_t (y_t - a \cdot t - b)^2, \quad t = 0, \ldots, n-1. \qquad (2)$$

The trend is labelled increasing if the slope exceeds 1 and decreasing if it is below -1; the tool refuses to predict with fewer than three months. The same fit per category gives a budget-overrun check. The health score is a weighted sum of six component scores, each clipped to [0, 100]:

$$H = 0.30\, s_{sav} + 0.20\, s_{exp} + 0.15\, s_{bud} + 0.15\, s_{cash} + 0.10\, s_{div} + 0.10\, s_{goal}. \qquad (3)$$

Here the savings-rate score is the savings rate divided by 20 %, the expense score the unspent share of income divided by 50 %, the budget score the share of budgets at or below 100 %, the cash-cushion score the months of expenses covered by the balance divided by six, the diversification score the number of distinct asset types divided by five and the goal score the mean goal completion; ratings are Excellent from 90, Good from 75, Fair from 60, Needs Improvement from 40 and Critical below. Budget alerts are raised at 75, 90 and 100 % of a category budget.

Four implementation decisions are worth stating with their reasons. The categoriser is a linear model on sparse features because it trains in milliseconds, can be retrained inside a request when a user corrects a label, and its decision can be traced to words; a transformer would need data and a GPU that a household application does not have. The forecast refuses to run on fewer than three months rather than guess. Import is a two-step flow: a preview parses the file, categorises blank rows and lists every skipped row as 'Row n' with the reason (unparsable date, non-numeric or non-positive amount, blank description), and only a second call commits it. Budget alerts are written to the notifications table by a check that runs only when the dashboard or the notifications list is read, not on a timer and not when a transaction is saved, so no scheduler process is needed but an alert appears only after the next such read, and goals and investments are kept as plain rows (target, saved amount, quantity, prices) so that their totals feed the health score.

The health score was added because a raw balance says little about resilience. Its six parts are explainable on their own: each has a plain-language meaning, the weakest part is named on the dashboard, and every part is computed from the user's rows, with a neutral value of 50 when there is nothing to measure (no budgets, no expenses or no goals) and 0 for diversification when there are no holdings. The same per-user scoping applies to the assistant, whose answers are prefixed by the amount computed and followed by a standing notice that they are informational and not financial advice.

D. Assistant, Data and Evaluation Protocol

The assistant routes a question by keyword to one of five handlers (can I save, reduce expenses, health score, where am I spending, spend on a named category) and answers from SQL aggregates of the user's own rows; other questions get a fixed help text. No external dataset is used. The demo account holds 68 fictional transactions over six months, built from a 34-row sample file plus synthetic earlier months, so every figure on screen is synthetic. Because the model has no train and test split, we evaluate it on a 198-description probe set written by the team before scoring and never used for training: 11 categories times three tiers of six, tier A phrases close to the seed, tier B upper-case statement strings that contain a seed keyword, tier C unseen merchants and phrases. This is not an independent benchmark. The forecast is evaluated by a one-step-ahead rolling backtest on the demo history and by a simulation of 2,000 users per setting (12-month level plus drift, Gaussian noise and a 10 % chance of a +50 % spike). The rest is covered by 38 pytest tests, 49 smoke checks, a 1,000-row import file with 84 injected bad rows, 26 assistant questions and response-time runs on 127.0.0.1. Cross-validation on 51 seeds is reported as leave-one-out [20].

IV. RESULTS

A. Expense Categoriser

Table II lists the headline measurements. On the probe set the shipped model scores 71.7 % accuracy (142 of 198) and macro-F1 0.761, against 51.5 % for the keyword table and 9.1 % for always answering Food. The overall figure hides a sharp split: 100 % on tier A, 97 % on tier B and 18.2 % on tier C. Of the 56 errors, 48 are labelled Transport: a description with no word from the seed list gives an all-zero feature vector, the decision then comes from the intercepts, and Transport has the highest, so the class precision of Transport is 0.27 while its recall is 1.0. The model never says

it does not know, although its probabilities could support that: keeping only predictions with a top probability of at least 0.15 retains 62.1 % of items at 94.3 % accuracy, but the application applies no threshold. Leave-one-out on the 51 seed pairs gives only 11.8 %, since each phrase's words occur nowhere else.

TABLE II
MEASURED RESULTS OF FINJARVIS

Measure	Result
Categoriser, 198 probe items: accuracy / macro-F1	71.7 % / 0.761
Accuracy by tier A / B / C	100 / 97 / 18.2 %
Keyword table / majority class (accuracy)	51.5 % / 9.1 %
Repeat merchant, before / after one correction	18.2 / 100 %
Unseen merchants after 48 corrections	21.5 %
Forecast MAPE, 3 backtest points: trend / mean of all / mean of last 3	28.4 / 21.4 / 24.8 %
Simulated MAPE, 3 months: trend / mean of all	22.5 / 14.9 %
Simulated MAPE, 7 months: trend / mean of all	16.9 / 17.8 %
Demo health score	90.4 (Excellent)
5,000 random profiles: mean score, range	60.3, 8.7-98.2
Assistant: answerable handled / off-topic declined	14 of 20 / 6 of 6
Import of 1,000 rows: bad rows skipped / rows per s	84 of 84 / 35.2
pytest / smoke checks passed	38 of 38 / 49 of 49

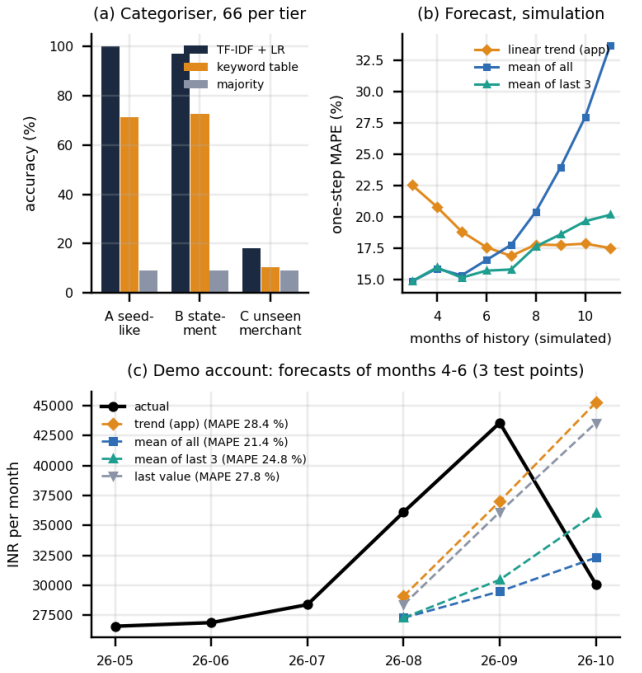


Fig. 3. (a) Categoriser accuracy by probe tier for the shipped model, the keyword table and the majority class; (b) simulated forecast error against months of history; (c) demo-account forecasts of months four to six by four methods.

Corrections help in one way only. When a user corrects one unseen merchant, the same merchant narrated three other ways is then labelled correctly 100 % of the time (from 18.2 %, 198 tests), but adding up to 48 random corrections moves accuracy on other new merchants only from 18.2 % to 21.5 %; the repeat test is easy because the variants share the corrected words. The 34-row sample file is labelled with 100 % accuracy, but its vocabulary overlaps the seed list, so that figure is optimistic.

Table III gives the per-category scores. Healthcare, Rent, Travel, Investments and Salary reach precision 1.0 because only tier-A and tier-B items are ever assigned to them, recall lies between 0.61 (Shopping) and 0.78 (Education) for every other class, and the single poor precision is the catch-all Transport. The remaining tier-B errors are informative: 'UPI-AMAZON-PAY-8821' goes to Entertainment because the seed list contains 'amazon prime video', and 'UPI-HOSPITAL-BILL' goes to Bills.

TABLE III
PER-CATEGORY SCORES ON THE PROBE SET (18 ITEMS EACH)

Category	P	R	F1	Category	P	R	F1
Food	0.92	0.67	0.77	Healthcare	1.00	0.67	0.80
Transport	0.27	1.00	0.43	Rent	1.00	0.67	0.80
Entertainment	0.87	0.72	0.79	Travel	1.00	0.72	0.84
Bills	0.93	0.72	0.81	Investments	1.00	0.67	0.80
Shopping	0.92	0.61	0.73	Salary	1.00	0.67	0.80
Education	0.82	0.78	0.80				

B. Forecast

On the demo account, with monthly expenses of 26,579; 26,878; 28,384; 36,118; 43,579 and 30,080 INR, we forecast months four to six from the months before them. The trend follows the August and September rise but overshoots October, and its MAPE of 28.4 % is worse than the mean of all history (21.4 %), the mean of the last three months (24.8 %) and repeating the last value (27.8 %); MAE is 9,597, 8,385, 9,301 and 9,565 INR. Three test points prove little, so we simulated 2,000 users per setting. At noise 0.10 the trend has a MAPE of 22.5 % with three months of history against 14.9 % for a plain mean, and it is better than the mean of all history only from about seven months (16.9 against 17.8 %); the mean of the last three months was at least as good as the trend in every noise setting (e.g. 28.2 against 30.8 % at 0.20). Across noise levels from 0.05 to 0.30 the trend's simulated MAPE rises from 14.0 to 54.0 %, the last-three-month mean from 13.1 to 51.1 %. The minimum of three months is therefore too low for this method. For November the application predicts 39,471 INR against a recent three-month average of 36,592 INR.

C. Health Score and Assistant

The demo account scores 90.4 (Excellent) with components 100, 100, 100, 100, 80 and 24; goal progress is the weakest. From a baseline profile of 80.7, raising the savings rate from 0 to 20 % lifts the score from 50.7 to 80.7 and then saturates; over 5,000 random profiles the score ranges from 8.7 to 98.2 (mean 60.3; 1425 Critical, 218 Excellent), and raising the savings rate by five points never lowered it. The weights are chosen by the authors, not fitted, and savings rate and expense ratio, which together carry half the score, are strongly correlated. The assistant handled 14 of 20 answerable questions; it missed 'groceries', 'biggest expense', 'eating out', 'largest purchase', 'run out of money' and 'which category to cut', which fell to the help text, and it declined all 6 off-topic questions. Its numbers are correct: 'How much did I spend on Food?' returns INR 17,365, equal to a direct sum, and the top category Rent matches a group-by. One quirk remains: 'Am I overspending on shopping?' is answered with the Shopping total, not a comparison with its budget.

The sensitivity runs show how the score behaves. With expenses at 20,000, 32,500, 45,000, 55,000, 65,000 and

75,000 INR a month the score falls from 85.5 to 80.7, 70.9, 63.7, 56.8 and 56.3, and a balance of 0, 50,000, 100,000 and 195,000 INR or more gives 73.0, 76.8, 80.7 and 88.0, capped at six months of expenses. Goal progress from 0 to 100 % and the number of asset types from 0 to 5 each move the score by 10 points (76.7 to 86.7), the full weight of the part. The score therefore rewards a high savings rate and low spending most, and these two inputs overlap, so a household with high income but no goals or holdings still scores in the 70s.

D. Tests and Response Time

TABLE IV
RESPONSE TIME IN MS, MEDIAN / P95 (127.0.0.1, ONE USER)

Endpoint	ms
Login (bcrypt verify)	305 / 367
GET dashboard	67 / 129
GET analytics	72 / 128
POST predict-category	7.9 / 20.5
POST transaction (auto-categorise)	15.2 / 20.2
POST assistant ask (analytics)	21.6 / 47.9
GET report CSV / PDF / Excel	18 / 136 / 209 (median)
Import preview, 100 rows	667 / 933

The 38 pytest tests (authentication 9, budgets 6, health score 6, ML 9, transactions 8) and 49 smoke checks pass, covering weak and wrong passwords, a tampered token, per-user isolation, CRUD, import, reports and validation. Ordinary calls take under 75 ms apart from login (bcrypt) and report generation. The import commit writes one row per database session and runs at 35.2 rows/s; of 1,000 rows, all 84 injected bad rows (bad dates, negative or non-numeric amounts, blank descriptions) were skipped and 916 imported with automatic categories.

E. Output Screens

Figures 4 and 5 were captured from the running application with the demo account. The dashboard shows the month's balance, income, expenses, savings, net worth and the health score of 90 with its six components; the assistant answers 'Where am I spending the most money?' and 'What is my financial health?' from the stored rows, each marked as calculated from the user's data.

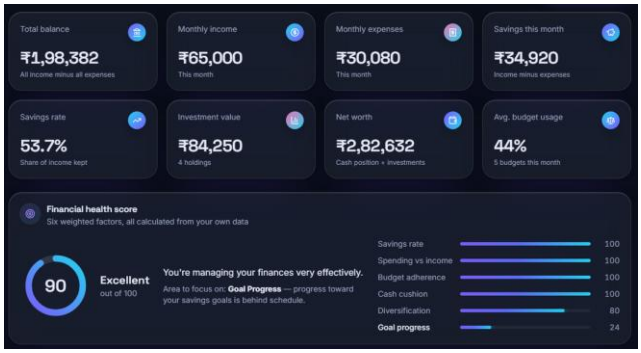


Fig. 4. Real screen: dashboard of the demo account with balance, income, expenses, savings rate, net worth and the financial health score (90, Excellent).

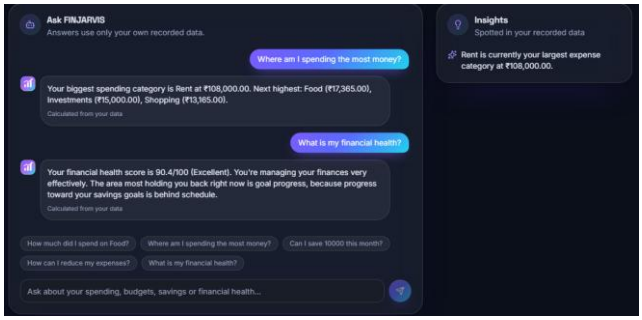


Fig. 5. Real screen: the assistant answering from the demo account's own transactions and health score.

F. Comparison with the Base Paper

The base paper reports 92.3 % categorisation accuracy with BERT on 50 categories, R2 0.63 for LSTM savings forecasts and 94 % F1 for anomalies [1]. FINJARVIS measures 71.7 % on 11 categories with a 51-example model, evaluates a trend forecast by MAPE on a different series, and has no anomaly detector, so no number is comparable: the datasets, label sets, models and targets differ, and the base data are not available. What can be said is qualitative: a context-free bag-of-words model fails on unseen merchants, which a pre-trained language model [12] is designed to reduce, and forecasting with few observations favours simple baselines [15]. In return the system is inspectable end to end, runs in milliseconds on a CPU and has been run and tested.

Threats to validity. The probe set was written by the team that built the model, so tier A and B reflect the vocabulary it knows; the demo data are fictional with a seeded gentle upward drift, which flatters a trend model; the forecast backtest has three points; latency was measured once on a busy desktop and moves between runs, while accuracy figures did not; and the assistant questions were written after reading its handlers. The numbers should be read as a description of this implementation, not as an estimate of performance on real households.

G. Failure Cases and Observed Limits

Unseen merchants are mislabelled silently, mostly as Transport (tier C 18.2 %). The seed list is tiny and corrections do not generalise. The trend forecast is not better than an average below about seven months. Anomaly detection (objective 4), time-series models beyond a linear trend, a reinforcement-learning budget recommender, a loan tracker and scheduled alerts are not implemented; budgets are set by the user and advice comes from rules. The abstract names Flask and MySQL, whereas the shipped system uses FastAPI and SQLite. Tokens are home-made bearer tokens with no revocation and a default secret if none is set, import commits row by row, and budget-alert de-duplication is by message text, so a budget that is crossed again in a later month raises no new alert. The data are fictional; there was no user study, concurrency test or security scan, and the language-model layer and live price refresh were not run.

V. CONCLUSION AND FUTURE SCOPE

FINJARVIS delivers accounts, entry and import of transactions, automatic categories, budgets with alerts, goals,

investments, a next-month forecast, a health score, an assistant and reports, verified by 38 tests and 49 end-to-end checks. Against the six objectives, accounts and import, categorisation, forecasting of expenses, budget alerts and the dashboard are achieved in simple form; forecasting of savings with time-series models and anomaly detection are not. The main finding is that the transparent models work where the input resembles their data (97 % on statement strings) and fail quietly where it does not (18.2 % on new merchants), and that three months are too little for a linear trend. The limits are the 51-example training set, synthetic data and the missing modules.

Future work: train the categoriser on a public or consented transaction corpus and abstain below a confidence threshold; require at least seven months, or switch to a recent average, for forecasts; add anomaly detection with a z-score or Isolation Forest rule and test it on injected outliers; fit the health-score weights to user outcomes; batch the import commit; and add refresh-token authentication and a user study.

REFERENCES

[1] R. Kharat, S. Solapurkar, A. Tikkas, M. Uttarwar and A. Yadav, "A smart AI-based personal finance assistant: A multi-model approach for expense categorization, budgeting, forecasting and anomaly detection," Int. J. Research in Applied Science and Engineering Technology (IJRASET), vol. 13, no. XI, pp. 167-180, Nov. 2025, doi: 10.22214/ijraset.2025.75019.

[2] A. Lusardi and O. S. Mitchell, "The economic importance of financial literacy: Theory and evidence," J. Economic Literature, vol. 52, no. 1, pp. 5-44, 2014.

[3] P. L. To, "Enhancing financial literacy and management through goal-directed design and gamification in personal finance application," arXiv:2601.08640, Jan. 2026.

[4] M. Bayer, "SQLAlchemy," in The Architecture of Open Source Applications, Volume II, A. Brown and G. Wilson, Eds. 2012.

[5] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[6] N. Provos and D. Mazieres, "A future-adaptable password scheme," in Proc. USENIX Annual Technical Conf., FREENIX Track, 1999, pp. 81-91.

[7] N. V. Chawla, K. W. Bowyer, L. O. Hall and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," J. Artificial Intelligence Research, vol. 16, pp. 321-357, 2002.

[8] C. J. C. H. Watkins and P. Dayan, "Q-learning," Machine Learning, vol. 8, no. 3-4, pp. 279-292, 1992.

[9] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735-1780, 1997.

[10] F. T. Liu, K. M. Ting and Z.-H. Zhou, "Isolation forest," in Proc. 8th IEEE Int. Conf. on Data Mining (ICDM), 2008, pp. 413-422.

[11] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Information Processing & Management, vol. 24, no. 5, pp. 513-523, 1988.

[12] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, 2019, pp. 4171-4186.

[13] D. Araci, "FinBERT: Financial sentiment analysis with pre-trained language models," arXiv:1908.10063, 2019.

[14] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," Int. J. Forecasting, vol. 22, no. 4, pp. 679-688, 2006.

[15] S. Makridakis, E. Spiliotis and V. Assimakopoulos, "Statistical and machine learning forecasting methods: Concerns and ways forward," PLoS ONE, vol. 13, no. 3, e0194889, 2018.

[16] V. Dilip and R. S. Janaki, "FinSenseAI: An explainable AI-based personal finance recommendation and expense prediction system using hybrid machine learning models," Int. J. Innovative Science and Research Technology, vol. 11, no. 7, pp. 1637-1650, Jul. 2026, doi: 10.38124/ijisrt/26jul842.

[17] F. R. Abiya, S. Parvathy, M. R. Devende, S. R. Ajmalsha and C. Viji, "Personal expense leakage detection and budget optimization using machine learning," Int. J. Research in Applied Science and Engineering Technology (IJRASET), Apr. 2026, doi: 10.22214/ijraset.2026.79467.

[18] A. Theerthala, "Synthesizing behaviorally-grounded reasoning chains: A data-generation framework for personal finance LLMs," arXiv:2509.14180 [cs.CL], Sep. 2025.

[19] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Univ. of California, Irvine, 2000.

[20] R. Kohavi, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in Proc. 14th Int. Joint Conf. on Artificial Intelligence (IJCAI), 1995, pp. 1137-1143.